

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – II

Roll No. : T045	Name:Omkar Thukarul
Class : TYIT	Batch: 1
Date of Assignment: 09/07/2026	Date/Time of Submission:09/07/2026

Aim:Write the program with different features of C#

A:Function Overloading

CODE:

Modelmvc.cs

```
using System;
public class ABC
{
    void Add(int num, int num1)
    {
        Console.WriteLine("Addition of 2: " + (num + num1));
    }
    void Add(int num, int num1, int num2)
    {
        Console.WriteLine("Addition of 3: " + (num + num1 + num2));
    }
    public static void Main(string[] args)
    {
        ABC obj = new ABC();
        obj.Add(18, 26);
        obj.Add(18, 26, 29);
    }
}
```

OUTPUT:-

```
Addition of 2:44
Addition of 3:73

=== Code Execution Successful ===
```

B:Inheritance (all types)

CODE:

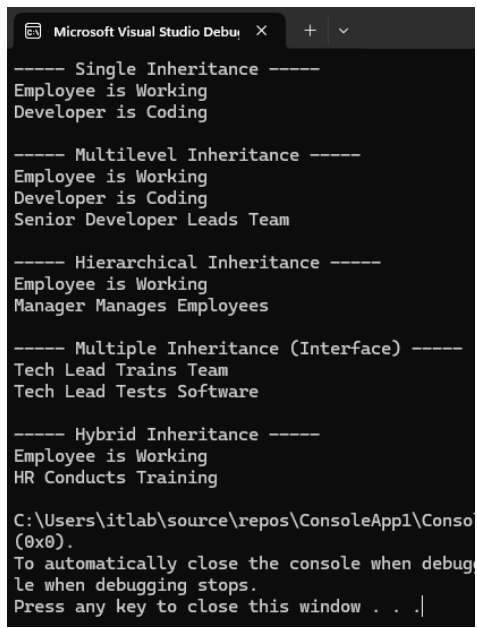
```
using System;
// Base Class
class Employee
```

```
{
    public void Work()
    {
        Console.WriteLine("Employee is Working");
    }
}
// Single Inheritance
class Developer : Employee
{
    public void Code()
    {
        Console.WriteLine("Developer is Coding");
    }
}
// Multilevel Inheritance
class SeniorDeveloper : Developer
{
    public void Lead()
    {
        Console.WriteLine("Senior Developer Leads Team");
    }
}
// Hierarchical Inheritance
class Manager : Employee
{
    public void Manage()
    {
        Console.WriteLine("Manager Manages Employees");
    }
}
// Multiple Inheritance using Interfaces
interface ITrainer
{
    void Train();
}
interface ITester
{
    void Test();
}
class TechLead : ITrainer, ITester
{
    public void Train()
    {
        Console.WriteLine("Tech Lead Trains Team");
    }
}
```

```
    }

    public void Test()
    {
        Console.WriteLine("Tech Lead Tests Software");
    }
}
// Hybrid Inheritance
class HR : Employee, ITrainer
{
    public void Train()
    {
        Console.WriteLine("HR Conducts Training");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("----- Single Inheritance -----");
        Developer d = new Developer();
        d.Work();
        d.Code();
        Console.WriteLine("\n----- Multilevel Inheritance -----");
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.Work();
        sd.Code();
        sd.Lead();
        Console.WriteLine("\n----- Hierarchical Inheritance -----");
        Manager m = new Manager();
        m.Work();
        m.Manage();
        Console.WriteLine("\n----- Multiple Inheritance -----");
        TechLead tl = new TechLead();
        tl.Train();
        tl.Test();
        Console.WriteLine("\n----- Hybrid Inheritance -----");
        HR hr = new HR();
        hr.Work();
        hr.Train();
    }
}
```

OUTPUT:-



```
Microsoft Visual Studio Debug Console
----- Single Inheritance -----
Employee is Working
Developer is Coding

----- Multilevel Inheritance -----
Employee is Working
Developer is Coding
Senior Developer Leads Team

----- Hierarchical Inheritance -----
Employee is Working
Manager Manages Employees

----- Multiple Inheritance (Interface) -----
Tech Lead Trains Team
Tech Lead Tests Software

----- Hybrid Inheritance -----
Employee is Working
HR Conducts Training

C:\Users\itlab\source\repos\ConsoleApp1\ConsoleApp1(0x0).
To automatically close the console when debugging stops,
press any key to close this window . . .|
```

C:Constructor overloading

CODE:

using System;

public class ABC

{

public ABC(int num, int num1)

{

Console.WriteLine("Addition of 2 numbers = " + (num + num1));

}

public ABC(int num, int num1, int num2)

{

Console.WriteLine("Addition of 3 numbers = " + (num + num1 + num2));

}

public static void Main(string[] args)

{

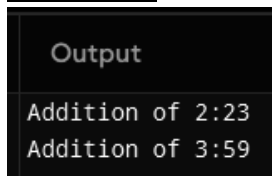
ABC obj = new ABC(11, 12);

ABC obj1 = new ABC(11, 12, 13);

}

}

OUTPUT:-



```
Output
Addition of 2:23
Addition of 3:59
```

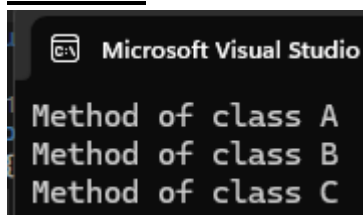
D:Interfaces

CODE:

```
using System;
public interface A
{
    void Fun1();
}
public interface B
{
    void Fun2();
}
public class C : A, B
{
    public void Fun1()
    {
        Console.WriteLine("Method of class A");
    }
    public void Fun2()
    {
        Console.WriteLine("Method of class B");
    }
    public void Fun3()
    {
        Console.WriteLine("Method of class C");
    }
}
public class ABC
{
    public static void Main(string[] args)
    {
        C obj = new C();

        obj.Fun1();
        obj.Fun2();
        obj.Fun3();
    }
}
```

OUTPUT:



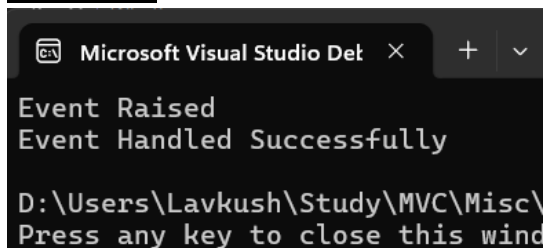
E:Using Delegates and events

CODE:

```
using System;
public delegate void Notify();
class EventDemo
{
    public event Notify MyEvent;

    public void Show()
    {
        Console.WriteLine("Event Raised");
        MyEvent();
    }
}
class Program
{
    static void Message()
    {
        Console.WriteLine("Event Handled Successfully");
    }
    static void Main(string[] args)
    {
        EventDemo obj = new EventDemo();
        obj.MyEvent += Message;
        obj.Show();
    }
}
```

OUTPUT:-



```
Microsoft Visual Studio Det
Event Raised
Event Handled Successfully
D:\Users\Lavkush\Study\MVC\Misc\
Press any key to close this window
```

F:Exception handling.

CODE:

```
using System;
class Program
{
    static void Main(string[] args)
    {
        try
        {
            int a = 10;
```

Omkar T045

```
        int b = 0;
        int c = a / b;
        Console.WriteLine(c);
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine("Cannot divide by zero.");
    }
    finally
    {
        Console.WriteLine("Program Ended.");
    }
}
```

OUTPUT:-

